

Django 教學 12

購物車與流程測試

第 18 章

從最小範例到專案實作與驗收

本份學習路線

先完成 [11_CRUD與物件權限](#)。

- **第 18 章：購物車工作流程與測試**

每章依序：概念、最小範例、語法、專案對照、實作與驗收。

[全課目錄](#) · [實作手冊對照](#)

第 18 章

購物車工作流程與測試

驗證購物車增改刪，測試 HTTP response 與資料庫狀態。

本章的操作環境與成果

在 LearnMart 的練習分支操作；LearnBoard 用來比較相同規則。

完成成果： 驗證購物車增改刪，測試 HTTP response 與資料庫狀態。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。
進階頁可回查，但所有基本驗收需完成。

18-1 從留言操作擴充到購物車

已完成的 LearnBoard 規則	本章 LearnMart 對應
作者由 request.user 指派	CartItem 的 user 由伺服器決定
操作前限制自己的留言	查詢先限制自己的 cart item
表單驗證後才儲存	數量、庫存、商品狀態都要檢查
成功後 redirect	POST 後回購物車或商品頁

本章完整練習 AAA、response assertion 與 database assertion。

18-2 CartItem 的資料規則

目前 LearnMart | 節錄／重排 | marketplace/models.py

```
class CartItem(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE,
                             related_name="cart_items")
    product = models.ForeignKey(Product, on_delete=models.CASCADE,
                                related_name="cart_items")
    quantity = models.PositiveIntegerField(
        "數量", default=1, validators=[MinValueValidator(1)]
    )
```

每一列代表「某 user 的某 product 與數量」。

18-3 Database constraint 防止重複 cart row

```
class Meta:
    constraints = [
        models.UniqueConstraint(
            fields=["user", "product"],
            name="unique_cart_product",
        )
    ]
```

View 的 `get_or_create()` 提供流程便利；database constraint 提供競爭情況下的最後一致性防線。

18-4 加入購物車必須是 POST

```
@require_POST
@login_required
def add_to_cart(request, pk):
    ...
```

這段與 checked-in source 順序一致。Decorator 由下往上套用：`login_required` 先包 function，`require_POST` 再成為最外層。

因此匿名 **GET** 先被外層 method 檢查擋成 405；匿名 **POST** 通過 method 檢查後才由 `login_required` 302 到 login。若交換兩個 decorator，匿名 GET 會先 redirect，而不是 405。

18-5 取得 Product 也要限制狀態

```
product = get_object_or_404(  
    Product,  
    pk=pk,  
    is_active=True,  
)
```

URL 中的 `pk` 是 client-controlled input。除了存在性，也要限制「可購買」狀態，不能只在 template 隱藏已下架商品。

18-6 解析 quantity：瀏覽器限制不是安全規則

```
try:
    quantity = max(1, int(request.POST.get("quantity", 1)))
except ValueError:
    quantity = 1
```

HTML 的 `min`、`max` 可被移除。伺服器仍要處理：

- 缺少值
- 非整數
- 0 或負數
- 大於庫存

目前實作將多種錯誤 fallback 成 1；正式 UX 可回傳更明確錯誤。

18-7 `get_or_create()` 有兩條路

```
item, created = CartItem.objects.get_or_create(  
    user=request.user,  
    product=product,  
)
```

- `created=True`：剛新增，數量設成提交值
- `created=False`：原本已有，累加但不超過庫存

回傳 tuple 的第二個值讓 View 判斷走哪條流程。

18-8 目前累加行為與提示有一個細節

```
item.quantity = (  
    quantity if created  
    else min(item.quantity + quantity, product.stock)  
)
```

若累加超過庫存，數量會靜默 cap 在 `product.stock`，接著仍顯示「已加入」。

教學觀察 規則正確不代表回饋最清楚；可在 workbook 練習加入明確提示。

18-9 Cart 顯示只取目前 user 的資料

```
items = request.user.cart_items.select_related(  
    "product", "product__seller"  
)  
total = sum(item.subtotal for item in items)
```

由 `request.user.cart_items` 反向關聯開始，天然限制 ownership；`select_related` 避免逐筆讀 product/seller 的額外 query。

18-10 更新 cart item 先縮小 ownership

```
item = get_object_or_404(  
    CartItem,  
    pk=pk,  
    user=request.user,  
)
```

只有 `pk=pk` 不夠。攻擊者可以改 URL id；加上 `user=request.user` 才能把物件範圍限制在自己的購物車。

18-11 更新與移除都使用 POST

```
@require_POST
@login_required
def update_cart(request, pk): ...

@require_POST
@login_required
def remove_from_cart(request, pk): ...
```

對應 template 的每個 form 都有：

- 明確 action URL
- method="post"
- CSRF token
- button，而不是改資料的普通 link

18-12 Context processor：每頁都需要 cart count

目前 LearnMart | 逐字摘錄 | `marketplace/context_processors.py`

```
def cart_count(request):  
    if request.user.is_authenticated:  
        return {  
            "cart_count": sum(  
                item.quantity  
                for item in request.user.cart_items.all()  
            )  
        }  
    return {"cart_count": 0}
```

它回傳 dictionary，合併到每個使用該 template backend 的 context。

18-13 Context processor 必須在 settings 註冊

```
"context_processors": [  
    "django.template.context_processors.request",  
    "django.contrib.auth.context_processors.auth",  
    "django.contrib.messages.context_processors.messages",  
    "marketplace.context_processors.cart_count",  
]
```

因此 `base.html` 在任何頁面都能顯示 `{{ cart_count }}`，不必每個 View 重複提供。

18-14 Context processor 的成本也會遍及每頁

目前實作對每個已登入 request 讀取 cart rows，再由 Python `sum()`。

可能改進方向：

```
from django.db.models import Sum

request.user.cart_items.aggregate(
    total=Sum("quantity")
)["total"] or 0
```

是否值得優化要看流量與 query；先知道「全域方便」也代表「全域成本」。

18-15 Mutation route matrix (1/2)

Route	GET	POST mutation gate
register	顯示 form	anonymous flow ; Form 建 User / hash password
logout	405	CSRF ; 結束 session (無額外 role gate)
product create	seller form	login + seller ; server 指派 seller
product update	owner seller form	login + seller + owner-scoped Product
add cart	405	login ; active Product ; server 指派 user
update/remove cart	405	login + CartItem owner

GET 可顯示 form，不代表 GET 執行 mutation。POST form 還需要 CSRF token / Origin policy；表中再列 application authorization。

18-16 Mutation route matrix (2/2)

Route	GET	POST mutation gate
checkout	unbound form + cart summary	login + user cart ; server 算 buyer/total/snapshot
shipping	405	login + seller + order membership
review	405	login + purchase/status eligibility ; server author
board create	authenticated form	login ; server author

目前 checkout 與 review 只要求 authenticated user，再以 cart/purchase rules 限制；**沒有 buyer-role check**。Product create/update 與 board create 則是 GET 顯示 form、POST 才修改資料。

18-17 開始寫流程測試：Arrange-Act-Assert

Arrange：建立 user、product、初始 cart state

Act：登入並 POST add-to-cart

Assert：response 與 database state 都正確

測試不是只看「頁面沒有 500」。重要 workflow 通常同時有 HTTP 結果與資料副作用。

18-18 TestCase 與真實 transaction 測試

```
class MarketplaceFlowTests(TestCase):  
    def setUp(self): ...
```

- Runner 為整次執行建立 test database，不是每個 method 建一個實體 database。
- TestCase 以外層 transaction 與 savepoint／rollback 快速隔離 methods。
- 因此一般 CRUD、response、view-level atomic rollback 很適合用 TestCase。
- 真實 commit boundary、blocking／locking 要用 TransactionTestCase，並搭配支援功能的 database、separate connections 與審慎的 concurrency test。
- SQLite 不能用來證明 row-level select_for_update() 行為。

18-19 setUp() 每個 test method 前執行

目前 LearnMart | 節錄／重排 | marketplace/tests.py

```
def setUp(self):  
    self.seller = User.objects.create_user(...)  
    self.buyer = User.objects.create_user(...)  
    category = Category.objects.create(...)  
    self.product = Product.objects.create(...)
```

每個測試從一致且獨立的資料狀態開始。

18-20 測試匿名 cart redirect

```
def test_anonymous_user_cannot_add_cart(self):  
    response = self.client.post(  
        reverse("marketplace:add-to-cart", args=[self.product.pk])  
    )  
    self.assertRedirects(  
        response,  
        f"/accounts/login/?next=/products/{self.product.pk}/cart/",  
    )
```

這個測試驗證身份層與 `next`，不是 CSRF。

18-21 測試 client 預設不強制 CSRF

```
self.client.post(url, data)
```

Django 一般 `Client()` 預設不執行 CSRF 檢查，方便測試 View 行為。因此成功 POST 測試不能證明 CSRF token 正確。

需要專門驗證時：

```
Client(enforce_csrf_checks=True)
```

並處理 cookie/token 流程。

18-22 測試登入後的 database state

```
def test_buyer_can_add_cart(self):
    self.client.login(username="buyer", password="safe-pass-123")
    self.client.post(
        reverse("marketplace:add-to-cart", args=[self.product.pk]),
        {"quantity": 2},
    )
    self.assertEqual(
        CartItem.objects.get(user=self.buyer).quantity,
        2,
    )
```

Act 後直接查 database，確認副作用而非只確認 redirect。

18-23 login() 與 force_login()

- `self.client.login(...)`：驗證 credentials，適合順便驗證密碼可登入
- `self.client.force_login(user)`：直接建立 authenticated session，當測試重點不是登入本身時更清楚

若使用 `login()`，可以先 assert return value，避免密碼打錯導致後續測試用匿名身份執行。

18-24 概念檢核

1. Database unique constraint 與 `get_or_create()` 各解決什麼？
2. 為何 cart lookup 必須加 `user=request.user` ？
3. Context processor 的便利與成本是什麼？
4. 一個 cart test 應同時 assert 哪些面向？
5. 一般 test client 的成功 POST 為何不能證明 CSRF ？

答案見配套實作手冊原第 4 章

18-25 LearnMart 實作

任務：加入「超過庫存不改變 cart」與「不能更新別人的 cart item」測試。

驗收重點：

- status／redirect 符合設計
- 原有 cart quantity 不變
- 不產生跨 user 修改
- full test suite 仍通過

LearnMart 測試步驟與參考片段；LearnBoard 對應練習

18-26 每個重要 POST 至少驗證兩件事

HTTP 行為：

```
self.assertRedirects(response, expected_url)
```

資料行為：

```
obj.refresh_from_db()  
self.assertEqual(obj.stock, 3)
```

只測 302 不代表資料真的正確。

18-27 `refresh_from_db()` 為什麼重要？

Python 手上的 instance 可能是舊值。

```
product.refresh_from_db()
```

重新從 DB 讀，才是在驗證「真正被寫入的結果」。

18-28 單支測試定位

```
uv run python manage.py test \
    marketplace.tests.CheckoutTests.test_checkout_creates_order_and_reduces_stock
```

先讓單支綠，再跑 class/app/all。

18-29 Test failure 常見類型

AssertionError	→ expected/actual 不同
IntegrityError	→ constraint / required relation
NoReverseMatch	→ URL name/args
DoesNotExist	→ fixture/setup 不完整
403/404/405 mismatch	→ auth/ownership/method 邏輯

先分類，再找檔案。

18-30 LearnBoard：把測試矩陣移到商城

LearnBoard 目前測試已固定這些規則：

LearnBoard 測試行為	LearnMart 對應問題
匿名不能開新增頁	匿名不能加入購物車／進入 checkout
建立留言時 author 是登入者	建立商品時 seller 是登入者
非作者編輯得到 404	非 owner seller 編輯商品得到 404
非作者刪除得到 403	buyer 出貨得到 403
staff 可刪除留言	seller 只能處理自己商品所在的訂單

每增加一條規則，都同時檢查 response、資料是否改變，以及失敗時的 side effects。

第 18 章實作與離堂檢核

任務： 驗證購物車增改刪，測試 HTTP response 與資料庫狀態。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

配套練習： [LearnBoard 02 原第 6 章](#)；[LearnMart 02 原第 4 章](#)。手冊保留原章號，對照表列出本課位置。

本份完成與後續

下一份：[13_訂單結帳與交易](#)。

- 保留本份操作紀錄，確認使用正確的專案與資料庫。
- 章節與實作對應可由 [全課目錄](#) 回查。